Shape Coverage Is a Boot-Time Cost: Compiled-Shape Padding in Production TPU and GPU Serving

Stack: vLLM 0.25.0 and `tpu-inference` 0.25.0 on a `v5litepod-4` TPU slice (JAX 0.10.2, tensor-parallel degree TP=4), with an NVIDIA L4 for the GPU control.

Abstract

Accelerator serving stacks round every step up to a compiled or captured shape, on the assumption that rounding up means paying for the shape rounded up to — the premise behind length bucketing, shape-aware admission control, and ladder design. We measure what is actually paid, on production TPU and GPU stacks, and **find it false on two of the three dimensions shape coverage quantizes.**

Per-request prompt-length padding **does not exist**. Request-slot padding is **free** — under 0.7 µs per padded slot against 27.5 µs if paid, because the attention kernel does no work for unused slots — and a GPU control replicates this at 67 µs (0.6% of the step) on the one dimension both architectures share; the token dimension is TPU-only. Token padding is **the exception**: real arithmetic, paid at 23.1% of nominal at batch 4 and indistinguishable from zero at batch 16.

On that dimension, ladder design still pays, and the effective variable is placement, not shape count. One entry added where the workload needs it cuts latency 12.1% at the straddling prompt length, at no memory cost, while a uniformly finer ladder buys nothing further and fails to boot near the stock memory fraction. Chosen offline from a length distribution, a placement predicts the gain to within 5% and beats BucketServe's own objective by 1.61 ms at equal shape count. The gain is a latency reduction below saturation (46% at the knee), not added capacity (2.6% at saturation), and shrinks from 12.3% to 1.7% once prefix caching is on.

What shape coverage costs is not run-time padding but a one-time boot cost, amortized by a persistent cache. Every number here regenerates from captured data by a script that fails on disagreement; the mechanisms proposed to explain them receive no equivalent check.

1 Introduction

A GPU serving stack resolves kernel shapes at runtime; a TPU stack cannot. XLA, the compiler that turns a TPU model into fixed-shape executables, compiles for fixed shapes, and recompiling per request is impossible at serving latencies, so vLLM's TPU backend precompiles a ladder of shapes and rounds every step up to one of them. vLLM's CUDA path does something structurally similar for a different reason: it captures one CUDA graph per batch size from a fixed set and pads a batch up to the next captured size.

Both designs create the same apparent inefficiency, and a literature has grown around removing it. **BucketServe** derives an optimal length-bucket boundary and then declines to compute it, describing it as *"computationally expensive to calculate in practice."* **LAPS** captures a graph per `(length, batch)` cell and notes that *"the number of graphs must be limited."* Both take for granted that the padding they manage is paid at run time.

This paper presents an empirical evaluation of run-time shape-padding overhead. We instrument the three quantized dimensions of a TPU serving stack, isolate the mechanism responsible for each, and repeat the central measurement on a GPU using the same framework and instrument. Run-time padding proves close to free on both, and the cost of shape coverage proves to be a boot-time charge that neither system measures.

A second theme shaped what was measured. Our measurements have proved durable and our explanations have not: no reported number has been withdrawn, while four of the last five retractions were claims about mechanism. §2 states the protocol adopted in response, and §6 reports what the asymmetry does and does not license.

Contributions.

1. **The premise, measured** (§4.1, §4.2, §4.9). Padding a batch up to a compiled or captured entry is close to free on a TPU ladder and on CUDA-graph capture alike; what shape coverage costs is paid at boot, not per step. The GPU comparison covers the request dimension, which is the dimension graph capture quantizes.

2. **The request ladder reported by the TPU stack differs from the one its attention kernel executes** (§4.1). This can be confirmed directly from the source code, by a paired hardware experiment, and from the compiler-emitted kernel name. It is absent from the RPA (Ragged Paged Attention) paper, from LENS, and from vendor documentation.

3. **A mechanism for free request padding** (§4.1): the ragged kernel performs under 0.7 µs of work

per padded slot, against 27.5 µs if it were paid, established by reducing the compiled slot count by a factor of 32 for a -0.9% change. The memory-bandwidth explanation that the same data invites is tested and rejected.

4. **Ladder design as placement rather than cardinality** (§4.3, §4.6): a fourteen-shape ladder placed against the workload reduces end-to-end latency by 12.1% at no memory cost, while a uniformly finer ladder achieves the same reduction and additionally costs 8.8% of cache capacity and 53% more startup. The choice can be made offline from a length distribution, predicting the measured result to within 5%.

5. **When ladder placement pays, and when it does not** (§4.4, §4.5, §4.6). The reduction is a latency gain below saturation rather than added capacity: median latency falls 46% just below the knee, while sustained throughput rises 2.6% at it. Enabling prefix caching, which production vLLM does by default, shortens the prefill onto a different compiled entry and reduces the same placement's benefit from 12.3% to 1.7%, so entries must be placed against uncached prefill lengths.

6. **A test of the measurements by construction** (§5). Four optimizations were designed against the premise and rejected, and their failures are what the measurements predict: those aimed at the request dimension or at per-request length padding cannot pay, because §4.1 and §4.2 find no cost there to recover.

Scope. This is primarily a measurement study. The single intervention it reports (§4.3) is set through a documented environment variable rather than a patch, so the measurement does not depend on the correctness of a modified scheduler.

2 Method

Controlled variables. Chunked prefill is **enabled** throughout, which is the stack's default and the configuration production runs; §4.2 reports a `-no-enable-chunked-prefill` control for the one result where it could be confounding. This matters for how the ladder results should be read. Under chunked prefill the scheduler selects a compiled shape for the *packed step*, not for an individual request, so a statement that a 1200-token prompt pads to 2048 describes what executes only while that prompt prefills alone in its step. At the two concurrent requests of §4.3 it does, and the evidence is internal to the measurement: two cells saving 512 and 1024 padded tokens agree on cost per padded token to within 1%, which co-scheduling would not produce, since a packed step would save a different amount in each cell. §4.4 measures where that ceases to hold and §4.6 restores it deliberately by staggering arrivals.

Prefix caching is disabled and asserted, except in §4.5 where it is the treatment. Chunked prefill, `max_model_len`, `max_num_batched_tokens`, tensor-parallel size, `gpu_memory_utilization`, `XLA_FLAGS` and `ATTN_BUCKETIZED_NUM_REQS` are recorded. Every run parses the server's own engine-configuration line and aborts if any controlled variable disagrees with the configuration it reports. The check has rejected three runs in which a controlled variable had drifted, and one in which a control was varied deliberately and declared.

Units. All timings are measured server-side, as differences between Prometheus histogram snapshots taken before and after each measurement block. Client-side wall-clock timings are not used, because they include round-trip time, HTTP overhead, tokenization and queueing delay.

Scope of instruments. A step-scoped property requires a step-scoped instrument. Single-step execution is verified per dispatch from the count delta of `iteration_tokens_total`, and dispatches that split are excluded rather than averaged. §6 notes that errors of this kind — an analysis that measures something other than its target — are the one class no check described here covers.

Request arrival. Where a measurement requires n requests to reach the scheduler within one step, they are released from a thread barrier after every connection is established, so arrival spread is measured in microseconds rather than milliseconds. §4.2 shows that this changes what is measurable.

Definitions. Three quantities are used throughout.

- **Flatness** — how far a short request's cost sits from what proportional scaling would predict, as a fraction of the distance to the full-bucket cost: `(cost(L) - p) / (cost(B) - p)`, where `p = cost(B) · L/B`. A value of 1.0 is a pure staircase, meaning cost is independent of true length and padding is fully paid; 0.0 is pure linearity, meaning cost is proportional to real tokens and padding is free.

- **Share of nominal padding paid** — `(measured - real) / (padded - real)`, where `real` is the cost ratio predicted by real tokens alone and `padded` the ratio predicted if the full compiled shape were paid. Zero means padding is free; one means it is fully paid, which is what the compiled-shape premise predicts.

- **Model rejection, quoted as a percentage** — the amount by which a candidate model's prediction exceeds the measurement, as a fraction of the prediction.

Table 1: The three quantized dimensions of the TPU serving stack, read from source.

	quantizes	ladder
D1	prompt length → prefill shape	*does not exist*
D2	scheduled tokens / step	`[16, 32, ..., 8192]`
D3	requests / step	`[8, ..., 256]` non-attention; `[256]` **attention**

Registered predictions. A proposed mechanism must state a falsifiable number before the measurement that would test it, recorded in the experiment's configuration file and therefore fixed before any result is seen. The requirement exists because a mechanism stated only in prose cannot be rejected by any check in this pipeline, whereas a number can. Three such predictions failed during this work, and each produced a result its confirmation would not have: the sharding ablation yielded a two-term cost model bounding per-step fixed overhead (§4.8), the model-scale ablation established that the regime map is set by batch size and dtype rather than by parameter count (§4.8), and the concurrency sweep identified that padding migrates from individual requests to the packed step under chunked prefill (§4.4).

Statistics. Medians over repeats, with 95% confidence intervals from 10,000 bootstrap resamples. Intervals are reported wherever a claim depends on the size of a difference, because §4.7 establishes that some cells are far noisier than others.

Models. Qwen3-4B is the primary model; SmolLM2-1.7B (head dimension 64, multi-head attention) and TinyLlama-1.1B (head dimension 64, grouped-query attention at 8:1) are used for the architecture contrast.

Traceability. Every run writes `meta.json` before doing work, records the configuration hash, git SHA and dirty flag, appends to a manifest, and is never overwritten. Numerical claims are tied to run identifiers and recomputed from captured data by `scripts/paper_numbers.py`; `./reproduce_all.sh` regenerates every number and figure and exits non-zero if any disagrees.

3 Three quantized dimensions

A serving step is rounded up to a compiled shape in three independent ways, and this paper keeps them separate because each has a different mechanism and a different cost. The ladders below are those the TPU backend constructs at startup and prints in its own warmup log, so they can be read off a running server rather than inferred:

Keeping them separate matters because they behave differently: request-slot padding turns out to be free, per-request length padding does not exist, and only token padding is paid. A measurement of one therefore says nothing about the others, and every result below states which dimension it concerns.

These ladders are properties of the scheduler rather than of the model. `get_token_paddings`, which builds the D2 ladder, takes a minimum size, a maximum size, and a gap; it receives no reference to the model or its configuration, and the D3 ladder is constructed the same way. A mixture-of-experts model is therefore given the same ladder as a dense one at equal `max_num_batched_tokens` and gap, and searching the backend for a per-step expert-capacity shape — a fourth quantized dimension, which a mixture-of-experts implementation might plausibly have introduced — finds none: the expert-side padding in that stack is a static weight layout fixed at load, not a shape the scheduler selects. Three dimensions is the complete list for both architectures.

This is read from the implementation rather than measured, which is the right kind of evidence for the claim: which shapes exist is a fact about the code, and no experiment could establish it more directly. It also bounds what architecture-generality can mean here. Whether these results transfer to another architecture is a question about cost per padded token, and never about which shapes that architecture is padded to.

4 Results

4.1 Request-dimension padding is free, and the mechanism is the data structure

`envs.ATTN_BUCKETIZED_NUM_REQS` defaults to `False`, and when it is off, `get_attn_req_paddings` returns `[max_req_size]`, a single bucket. Every boot prints both ladders, and they disagree:

```
Prepared request paddings: [8, 16, 32, 64, 128, 256]
Prepared attn request paddings: [256]
```

Executed request-slot count The attention kernel therefore executes at a fixed size of 256 request slots, independently of the batch size. Three measurements establish that this padding carries negligible cost.

The first is a paired experiment. Enabling the ladder for attention, so that it matches the one the stack advertises, changes decode by 0.0%: the two arms are identical to 0.1 ms at n=8 and n=9. This comparison excludes a large effect and supports nothing finer, since the paired difference is +0.00 ms with a 95% bootstrap interval of [-10.7, +10.8] ms at n=8, or ±20% of the decode phase.

Second, the compiler records the padding in the name it gives the kernel. The decode attention kernel is emitted

under a name that encodes the shapes it was compiled for, and the field holding the request-slot count reads 256 regardless of the batch actually being served. The stack therefore reports, in its own compilation output, that attention runs at 256 slots.

The third is an operator-level measurement. With prompt and output length fixed, so that per-sequence key–value state is constant, attention device time aggregated over a full 64-step generation is:

These figures alone do not discriminate between the hypotheses. The apparent argument — that a kernel doing work for 256 padded slots would be flat in n, that this is not flat, and therefore that padded slots are skipped — fails because flatness was never the alternative. Padded slots hold no key–value blocks, so no kernel could do work proportional to their state. The only cost padding can plausibly carry is a fixed per-slot overhead: walking 256 block-table entries, loading 256 metadata records, launching tiles across 256 slots regardless of occupancy. Such a cost is constant in n by construction. Fitting `T = a·n + b` through the endpoints (Table 3):

Residuals are under 2.3%, and the fixed term is 7 050 µs, or 42% of attention time at n=1, falling to 4% at n=16. If that term were a 256-slot padding cost it would amount to 27.5 µs per padded slot. The table is equally consistent with padded slots being skipped and with a fixed cost being paid for all 256, and so cannot establish either.

Discriminating experiment: reducing the slot count The discriminating experiment reduces the compiled slot count and observes the fixed term. Enabling `ATTN_BUCKETIZED_NUM_REQS` compiles attention at 8 slots rather than 256, a 32-fold reduction, profiled in both arms at the same real batch size:

A per-padded-slot cost would be expected to fall by approximately 97% when the slot count is reduced by a factor of 32. The measured reduction is 2%. The fixed term therefore represents block-table and dispatch overhead rather than padding. Stated as the bound the experiment supports: removing 248 of 256 slots moves the fitted fixed term by 167 µs, so a padded request slot costs **under 0.7 µs**, against the 27.5 µs per slot that fully-paid padding would imply. This is a factor of 41, and it is a bound rather than zero: at 256 slots the residue is up to approximately 170 µs, about 1% of attention time at n=1.

This measurement also runs the high-power version of the paired experiment above. That test used n=8 and n=9, where a fixed 256-slot cost would have been about 8% of attention time; at n=1 it would be 42%, and at n=1 the measured difference is -0.9%. The 0.0% result was not a low-power test concealing an effect, because the effect is absent where it would have been largest.

Testing the memory-bandwidth account The memory-bandwidth explanation is tested and rejected. A natural account of free request padding is that the step reads the whole weight set regardless of batch, so padded slots are absorbed into a fixed cost that batch size does not change. That account predicts utilization climbing toward the compute roof past the arithmetic-intensity ridge near 240 FLOP/byte, reaching approximately 49% at n=256. The quantity is **model FLOPs utilization (MFU)**: the arithmetic the model performs, divided by the arithmetic the chip could perform in the same time. A value of 5% means the chip spent 95% of its time doing something other than the model's floating-point work — waiting on memory, mostly. The table also reports utilization of high-bandwidth memory (HBM), the off-chip memory the weights are streamed from. Measured with `prompt_len=256` and `output_len=64`:

At n=256 the server has not admitted all 256 requests into the batch: some are still waiting in the queue, so the column does not describe a batch of 256 and no claim rests on it. It is shown because 256 is the batch size the prediction names. The refutation is stated at n=64, where queue time is 0.1 ms and the batch is genuinely full.

MFU here is `2 × parameters × tokens` against the chip's 197 TFLOP/s bf16 peak, with attention FLOPs excluded, which is the same dense weight-stationary accounting used for the intensity argument. Two caveats apply. MFU during memory-bound decode is close to a tautology, since it restates step time against a fixed numerator; and it is used here only to falsify a prediction expressed in the same units, not as a figure of merit.

A memory-bound step is one whose achieved bandwidth sits near the roof. This one falls monotonically to 21.4% by n=64, where the queue is 0.1 ms and the column is clean, so the account fails without recourse to the contaminated cells. Neither the compute-bound nor the memory-bound account is supported. One use of the roofline remains valid, namely byte accounting: the step reads 2.01 GB of weights regardless of batch size. Achieved bandwidth, however, is computed as bytes divided by measured time, and therefore restates the step time from which it is derived.

Promotion cost at the 8-to-16 request edge No promotion cost exists at the 8→16 request edge under the default configuration, because the attention kernel is already compiled at 256 slots and does not distinguish the two batch sizes.

Table 2: Attention device time aggregated over a 64-step generation, at fixed per-sequence key-value state.

n	1	2	4	8	16
attention (µs)	16 830	26 011	45 919	85 103	**163 535**
per real request	16 830	13 005	11 480	10 638	10 221

Table 3: Fitting the attention table to a linear model with a fixed term.

n	1	2	4	8	16
measured	16 830	26 011	45 919	85 103	163 535
9 780·n + 7 050	16 830	26 610	46 171	85 292	163 535

4.2 Token-dimension padding is paid, and its share falls with batch size

Token padding is a different quantity with a different mechanism. A prefill step carries hundreds to thousands of tokens, and padded tokens are real floating-point operations, because the kernel computes them. Token padding is therefore paid wherever the step's cost is dominated by arithmetic on those tokens.

Table 6 reports the share of nominal padding paid at a compiled boundary, with batch size fixed and sequence length held near-constant:

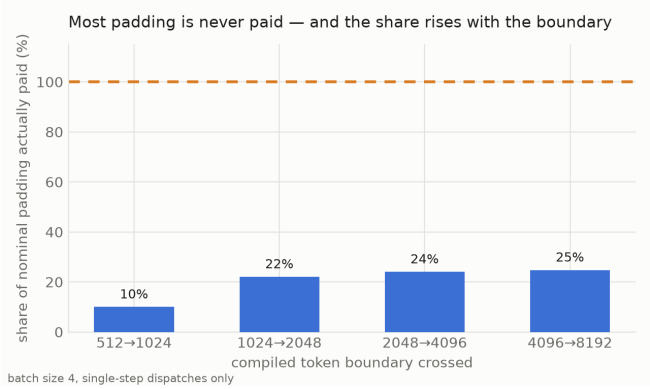

Figure 1: Share of nominal padding actually paid at each compiled boundary, against the 100% the compiled-shape premise predicts.

Confounding across ladder boundaries These rows are confounded, because each was measured over a different set of ladder boundaries. At fixed n=4 the paid share rises with boundary size, from 10.0% at 512→1024 to 24.8% at 4096→8192, so the particular boundaries a row contains shift its value independently of batch size. The sets differ: n=8 lacks 512→1024, the lowest-paying boundary, and n=16 lacks 4096→8192, the highest. Both omissions push in the same direction and exaggerate the decline. Comparing rows measured over different boundary sets is the most common error in this work, and it occurred here in a headline table.

Restricted to the two boundaries present in every row (Table 7):

The decline remains after matching, though the n=8 mean falls from 14.3% to 10.6%. An interval bootstrapped over two boundaries would not be meaningful, so the matched table establishes the ordering of the rows rather than their absolute values. The defensible statement is ordinal rather than monotone: the paid share is substantial at n≤2, intermediate at n=4–8, and indistinguishable from zero at n=16, with only the n=4 against n=16 contrast separated at interval level.

The n≤2 row rests on a single boundary, which is not one of the matched pair, and carries no interval. It is simultaneously the largest figure in the paper and the least well supported. The n=8 value of 14.3% excludes split dispatches; including them pools partial steps into the median and yields 16%, and the exclusion is the correct treatment.

Measurable range and the request launcher The upper limit of what can be measured was partly a property of the measurement apparatus. A naive launcher makes this quantity appear unmeasurable above n=8, because the scheduler splits every dispatch. Splitting tracks request count rather than token count — a dispatch of 8192 tokens at `max_num_batched_tokens=8192` never splits, while one of approximately 1024 tokens at n=8 splits half the time — which is the signature of an arrival race rather than a capacity limit. Releasing all requests from a thread barrier after connection setup reduced arrival spread by a factor of 7.6 at n=32, from 15.4 ms to 1.7 ms:

The real barrier lies between 16 and 32 rather than at 8. Under a synchronized launch n=16 becomes measurable, and the paid share there is indistinguishable from zero across three boundaries that each double the padded token count.

Paid share against boundary size The share paid rises with the size of the boundary. At fixed n=4 it is 10.0% at 512→1024, 22.1% at 1024→2048, 24.0% at 2048→4096 and 24.8% at 4096→8192. Batch size is held constant across these figures, so the comparison is between boundaries and not between load points.

Per-request length padding Per-request length padding does not exist. Holding batch size and total

Table 4: Attention device time with the compiled slot count reduced 32-fold. A per-slot padding cost would fall by about 97%; it falls by 2%.

n	flag off (256 slots)	flag on (8-slot ladder)	change
1	16 898 µs	16 749 µs	**-0.9%**
2	26 130 µs	25 999 µs	-0.5%
8	85 081 µs	85 055 µs	-0.0%
fitted fixed term `b`	7 158 µs	6 991 µs	**-2%**

Table 5: Utilization against batch size, testing the memory-bandwidth account of free request padding. The account predicts about 49% MFU at n=256.

n	1	8	32	64	256
MFU	0.3%	1.7%	3.6%	4.4%	**5.1%**
high-bandwidth memory (HBM) utilization	61.4%	52.1%	31.2%	21.4%	**11.1%**

tokens fixed and varying only the spread of request lengths, the batch-padding model is rejected in every ragged cell by at least 44%, and cost tracks packed tokens instead. The rejection range extends to 618%, but a span of more than an order of magnitude is a directional refutation rather than a measured effect size, so the minimum is the figure that carries the claim. Uniform controls, in which all candidate models agree, match to 1.9%. This is not an artifact of chunked prefill: with `-no-enable-chunked-prefill` the result is unchanged, the packed model winning 8 of 10 ragged cells and batch padding being rejected by 75–579%.

Padded share under four length distributions Padded share is governed by where a distribution's mass falls relative to the compiled boundaries, and not by its dispersion. Four prompt-length distributions were served at a single Poisson arrival rate (8 req/s, `output_len=64`, 120 requests each), reporting time to first token (TTFT) and inter-token latency (ITL). Their parameters were solved so that every family has the same mean length of 1000 tokens, so equal request rate means equal offered tokens and distribution shape is the only quantity that differs:

The padded share spans 8.5% to 33.3%, a factor of four, at equal offered tokens.

Dispersion as a predictor of padding Coefficient of variation does not order it. The three dispersed families run CV 0.58, 0.91 and 1.51 against padded shares of 27.4%, 33.3% and 28.0%, which is not monotone. Raggedness is therefore not the variable that governs padding, which is the assumption the bucketing literature rests on. What governs it is where the distribution's mass sits relative to the compiled boundaries: the fixed-length arm pads least because 1000 tokens sits just below the 1024 entry, so little is rounded away, and the same family would pad far more at 1100.

A retracted ordering A prediction registered before this measurement was wrong, and its failure retracts the explanation that motivated it. The same four families measured without matching offered tokens — mean lengths of 256, 384, 704 and 2056, so the uniform arm carried about eight times the tokens of the fixed arm — placed the fixed-length family highest, at 51.0% against 27.3% for uniform. The explanation offered for that ordering was that a fixed length just above a boundary pads every step by the same large amount while a spread distribution averages across buckets, and the prediction registered here was that the ordering would survive matching. It does not: at equal offered tokens the fixed family pads least of the four. The ordering followed from unequal token load, and the explanation built on it does not stand.

This is the conclusion of §4.3 reached from the other direction. There, ladder *placement* rather than shape count determines what a ladder buys; here, distribution *position* rather than dispersion determines what a workload pays. Both identify the same governing quantity: where lengths fall relative to boundaries.

Recoverable headroom, reported as two factors In place of a recoverable-headroom figure we report its two factors separately. Those are the padded share of executed tokens, which the table above establishes for four synthetic families and which remains workload-specific, and the paid share at a given batch size, reported with intervals earlier in this section. Multiplying them yields a headroom figure of roughly 4–9% of execution, and that product is invalid: the paid share moves with batch size, so a product formed from one workload's padded share and one batch size's paid share describes no configuration that was run.

4.3 Ladder design is placement, not cardinality

Token padding is the one dimension on which the premise survives, so it is the dimension an intervention should target. `VLLM_TPU_BUCKET_PADDING_GAP` provides the lever: unset, the TPU backend compiles ten expo-

Table 6: Share of nominal token padding paid, by batch size. Rows are confounded: each was measured over a different set of ladder boundaries.

batch size	median	mean	95% CI over boundaries	boundaries	clean dispatches/arm
1–2	~**85%**	—	*not computed*	1	see below
4	23.1%	20.2%	[13.5%, 24.4%]	4	9–15
8	14.3%	11.8%	[0.2%, 21.0%]	3	3–5
16	**-2.7%**	-5.9%	[-15.4%, +0.5%]	3	7–11

Table 7: Paid share restricted to the two boundaries present in every row.

n	1024→2048	2048→4096	mean
4	22.1%	24.0%	**23.1%**
8	0.2%	21.0%	**10.6%**
16	-2.7%	+0.5%	**-1.1%**

nentially spaced token shapes; set to 512, it compiles twenty-one linearly spaced ones. No patch is required.

To separate the effect of the ladder from differences between server instances, the design includes control prompt lengths. Of four prompt lengths, two pad to the same compiled shape on both ladders and two pad to a smaller shape on the fine ladder only, so the placebo cells measure everything that differs between two server boots except padding. Qwen3-4B, `v5litepod-4`, TP=4, two concurrent requests, `output_len=32`, 18 repeats per cell pooled over both arm orders:

The two treated cells agree on cost per padded token to within 1%, at 34.9 and 35.3 µs, while the two placebo cells differ by +0.5 ms in total. This agreement is the principal evidence for the effect. An arm-level offset, such as one server instance running uniformly slower, would produce a constant difference between the arms and therefore a per-token figure inversely proportional to the number of tokens saved; a padding effect scales with tokens saved, which is what the measurements show. For scale, a real prefill token costs 46.6 µs on the same arm (1200→3000 tokens for 83.9 ms), so at two concurrent requests a padded token costs about three quarters of a real one. Expressed as percentages, the twenty-one-shape ladder reduces end-to-end latency by 8.7% at prompt 1200 and 12.5% at prompt 3000.

Cost of the finer ladder The cost of the finer ladder is startup and memory headroom, not cache capacity. Measured with the compiled-shape cache cleared before each boot:

The warmup figures are cold. A persistent compilation cache amortizes them across restarts, and warm boots of the same two ladders measured 165–315 s, reflecting cache reuse rather than ladder size.

At a memory fraction of 0.85 the two ladders report the same 335,104 tokens, to the token. That equality also bounds any residual effect, since capacity is block-quantized and the 49 MB difference in executables would amount to roughly 1,300 tokens, or 81 blocks, and would have been visible. What the finer ladder costs is the backoff it forces: the ten-shape ladder runs at the 0.92 default for 367,360 tokens, the twenty-one-shape ladder does not start above 0.85, and the difference between those operating points is 32,256 tokens, or 8.8% of capacity.

That figure required measuring the cliff rather than assuming it. Configuring twenty-one shapes triggers an out-of-memory error (`RESOURCE_EXHAUSTED`) during warmup at memory fractions of 0.92, 0.90 and 0.88, and requires a reduction to 0.85 before the server starts. At 0.92 the allocation requests 32.50 MB against 12.40 MB free, and the failure is byte-identical across two independently provisioned hosts. A shortfall of 20 MB might suggest that a small reduction would suffice; it does not, since the requirement persists through three successive reductions.

The mechanism is not steady-state competition between executables and cache, which the identical capacities rule out. Every failed boot records a key–value cache size before terminating — 358,144 tokens at 0.90, 348,928 at 0.88 — so the cache is sized to fill the fraction first, and compilation then requests scratch memory against what remains. Shape coverage is charged to transient compilation headroom that the sizing step does not reserve, which is why the cost appears as a boot cliff rather than as a smaller cache.

Separating placement from cardinality The benefit, however, is bought by a different variable than the cost. The padding-gap lever changes the spacing law and the shape count together, so the measurements above attribute a benefit and a price to the same parameter without separating them. Every cost scales with cardinality: warmup, resident executables, and the headroom that sets the boot cliff. Two intermediate gaps separate them (Table 12). The default, gap-1024 and gap-2048 rows are measured together, at 0.92, in one server boot; the gap-512 row is not remeasured here and repeats the twenty-one-shape values already established in Table 10, from a different boot the day before:

† Reused from Table 10's boot, not this one. That boot's own default arm read 206.0 ms at prompt 1200 and 289.9 ms at prompt 3000 — 9.4 ms and 7.3 ms below this boot's default row for a nominally identical configu-

Table 8: Fraction of dispatches the scheduler splits, before and after synchronizing request release.

n	split under the old launcher	split under a synchronized launch
4	0%	0%
8	20%	**0%**
16	100%	**60%**
32	100%	100%

Table 9: Padded share under four prompt-length distributions matched on mean length, so that offered tokens are equal across arms. CV is the coefficient of variation of the sampled lengths.

length distribution	CV	padded share	TTFT p50 / p95	ITL p50 / p95
fixed-1000	0.00	**8.5%**	30.0 / 147.5 ms	4.9 / 9.0 ms
uniform	0.58	27.4%	45.6 / 74.7 ms	5.3 / 7.0 ms
lognormal	0.91	**33.3%**	43.6 / 253.3 ms	5.1 / 10.2 ms
bimodal	1.51	28.0%	17.1 / 114.2 ms	4.6 / 7.2 ms

Table 10: Ten-shape against twenty-one-shape ladder at two concurrent requests. Prompts 300 and 600 pad identically on both ladders and act as controls.

prompt	pads to (10 shapes)	pads to (21 shapes)	tokens saved	10-shape	21-shape	difference [95% CI]
300	512	512	0	149.8 ms	150.6 ms	+0.8 ms [+0.1, +1.6]
600	1024	1024	0	168.5 ms	168.6 ms	+0.1 ms [-0.5, +0.6]
1200	2048	**1536**	512	206.0 ms	188.2 ms	**-17.9 ms** [-18.2, -17.5]
3000	4096	**3072**	1024	289.9 ms	253.7 ms	**-36.2 ms** [-36.5, -36.0]

Table 11: What the finer ladder costs: startup, resident executables, and the memory fraction at which the server will start.

	10 shapes	21 shapes
cold warmup	285 s	436 s (+53%)
compiled-shape cache on disk	43 MB	92 MB
highest memory fraction that boots	0.92 (the default)	**0.85**
key–value cache at 0.92	367,360 tokens	*does not boot*
key–value cache at 0.85	335,104 tokens	335,104 tokens

Table 12: Separating placement from cardinality. Shape count ranges from 10 to 21 while the benefit follows only whether an entry falls between the prompt and the next default entry. Gap 512 is carried over from Table 10 rather than measured in this boot; see the note below the table.

ladder	shapes	1200 pads to	3000 pads to	prompt 1200	prompt 3000
default	10	2048	4096	215.4 ms	297.2 ms
gap 1024	14	2048	**3072**	210.9 ms	**256.5 ms**
gap 512 †	21	**1536**	**3072**	188.2 ms	253.7 ms
gap 2048	11	2048	4096	214.1 ms	296.1 ms

ration, in line with the 4–11 ms between-instance offsets Table 14 measures directly. Any comparison that mixes the gap-512 row with a value from this table's own boot carries that much additional, uncorrected uncertainty; the -12.1% figure below does not, since default and gap-1024 are both this boot's own measurements.

Against the ten-shape ladder, and correcting by the prompt-300 cell, the fourteen-shape ladder is +0.2 ms at prompt 1200 and -36.0 ms, or -12.1%, at prompt 3000 — both figures from arms measured in this same boot. It gains exactly where it places an entry the default lacks, at 3072, and nowhere else: it has no entry at 1536, so prompt 1200 is unaffected. The eleven-shape ladder places nothing new near either prompt and tracks the default at both. Shape count ranges from 10 to 21 across these arms while the benefit depends only on whether a boundary falls between the prompt and the next default entry.

Feasibility at the default memory fraction The fourteen-shape ladder boots at the stock 0.92 with 367,360 tokens, the same capacity as the default. The 8.8% is therefore the price of the twenty-one-shape ladder rather than of the optimization: a 12.1% reduction at prompt 3000 is available at full memory, for the warmup of four additional shapes. The twenty-one-shape ladder buys a further 8.7% at prompt 1200, and that increment is what costs 8.8% of capacity and 53% more startup.

The recommendation is therefore not to compile more shapes but to **compile the shapes the workload's prompt distribution straddles**. Cardinality is what the stack charges for; placement is what latency responds to. A badly placed boundary is an executable that the compiler pays to produce and the scheduler never uses.

4.4 The benefit does not decay with concurrency, contrary to a registered prediction

Because padding is paid at small batch and free at large (§4.2), the benefit of §4.3 appears as though it should be conditional on load. That reasoning predicts a crossing, and it was registered before measurement: the difference should shrink monotonically and reach zero between n=4 and n=16. Sweeping concurrency from 1 to 16 on both ladders, in both arm orders, does not support it. Placebo-corrected differences in milliseconds, negative where the finer ladder is faster:

There is no crossing. End-to-end latency is 3.5% to 12% lower at every concurrency sampled, the effect is non-monotone rather than decaying, and at n=16 it is larger in absolute terms than at n=1.

Why the prediction failed The reason the prediction failed is visible in what the stack executes. The prediction reasoned about an individual request's padding shrinking as more requests share a step. Under chunked prefill the scheduler admits requests in waves and packs them, and the size of the packed step, rather than any request's length, selects the compiled shape. Snapshotting the `iteration_tokens_total` histogram around one n=16 cell shows a single repeat costing about three prefill steps, landing in (256,512], (512,1024] and (2048,4096], alongside 93 decode steps of at most 16 tokens. The two arms produce near-identical step distributions, differing only in which compiled shape those steps round up to. Padding is therefore transferred from individual requests to the packed step rather than eliminated.

Tension with the paid-share curve This is in tension with §4.2, which finds the paid share falling to indistinguishable from zero by batch 16. The two measurements differ in what is held fixed: §4.2 varies batch size at a fixed boundary and attributes padding per request, while this sweep varies offered concurrency and allows the scheduler to choose step composition. If both are correct, the reconciliation is that per-request padding vanishes while per-step padding does not, and that a ladder acts on the second. Testing this requires padded tokens per packed step, which the available instrument cannot supply: `iteration_tokens_total` bins on powers of two, which is coarser than the ladder spacing under comparison. A step recorded in (2048,4096] pads to 2048, 2560, 3072, 3584 or 4096 depending on where in that bin it falls, and the two ladders differ precisely inside the bin.

A defect in the control cell One design defect bounds these results. Prompt 300 was intended as a placebo at every concurrency, on the reasoning that it pads to 512 on both ladders. The step histogram shows this holds only while each request prefills in its own dispatch: at $n \geq 4$ the packed step lands where the ladders differ, so the cell is treated rather than inert. Its measured difference is correspondingly unstable across arm orders at n=8 and n=16 (-6.0 against -27.7 ms, and -1.5 against -17.2 ms) where the treated cells are stable. The $n \geq 4$ figures above are therefore floors on the effect rather than unbiased estimates, and arm order is their only control.

An isolated-dispatch control The defect above is that concurrency does two things at once: it makes the regime realistic, and it lets the scheduler co-schedule prefills. Only the second breaks the control. Releasing requests on a 120 ms stagger separates them: a 3000-token prefill takes roughly 90 ms here, so each prefill runs alone in its step, while a 32-token decode takes about 160 ms, so requests still overlap in decode and the concurrency is real.

Isolation was verified rather than assumed. Scraping

Table 13: Placebo-corrected latency difference against concurrency, in milliseconds. Negative values favour the finer ladder.

prompt	n=1	n=2	n=4	n=8	n=16
1200	-9.9	-18.0	-8.9	-16.2	-23.0
3000	-18.7	-37.3	-21.5	-50.4	-37.0

the step-size histogram around eight staggered requests at prompt 300 shows **eight separate prefill steps, all in the (256, 512] bin**; the same eight released as a burst produce three steps, in (256,512], (512,1024] and (1024,2048]. The stagger therefore restores the per-request ladder mapping the control depends on.

A registered prediction that the control cell would return to zero was wrong: it shows a stable offset of 4 to 11 ms. Because 300 tokens pads to 512 on both ladders once prefills are isolated, and the histogram confirms they are, no ladder effect can reach that cell, so the offset is a difference between server instances and is exactly the quantity a control exists to measure. Subtracting it, prompt 1200 sits at zero as the ladders predict, since both pad it to 2048, and prompt 3000 carries a large reduction that does not decay through n=16. Intervals here are ±0.7 ms rather than floors.

The effect under isolation is substantially larger than under burst arrival, at 66 to 108 ms against the 18 to 50 ms of the table above. That is the same mechanism seen from the other side: a packed step amortizes one padding charge across several requests, while an isolated prefill pays its own in full. **The claim that the benefit does not decay through n=16 therefore holds at interval level under isolated dispatch, and as a floor under burst arrival.**

The sign survives this; the magnitude does not. The placebo's spread across arm orders at n=8 and n=16 is approximately 20 ms, comparable to several treated differences in the same table, so no n≥4 magnitude is resolvable at interval level. What is resolvable is direction: the raw differences at n=16 are -32.2 and -46.3 ms, and subtracting even the most extreme placebo estimate observed leaves both negative in both arm orders. The absence of a crossing is established at n=1 and n=2, where the placebo is valid; above that it is an observation with a floor.

4.5 The gain is a latency reduction below saturation, not additional capacity

The results above are latencies at fixed, small concurrency, while the cost of a longer ladder is denominated in cache tokens. A deployment choosing whether to spend memory on shapes requires the conversion. Table 15 sweeps offered load open-loop against both ladders, at the stock 0.92 fraction where both boot, with 60 requests per rate at prompt 3000:

Both ladders knee between 8 and 12 req/s and saturate near 14. Sustained goodput rises from 14.03 to 14.40 req/s, or 2.6%, so the padding removed was not entirely slack. But 2.6% is far short of what "padded tokens are real arithmetic" implies for a 25% reduction in the prefill shape, and it is not where the effect is concentrated. The effect is concentrated just below the knee: at 8 req/s the placement ladder is 46% faster at p50 and 39% faster at p95.

That shape is consistent with §4.2 and repairs part of the tension in §4.4. A saturated server packs prefills to the `max_num_batched_tokens` budget, so padding is amortized across a full step and the ladder has little influence, which is what the paid-share curve predicts. Below saturation the steps are smaller and closer to per-request, and the ladder entry accounts for most of the step's cost. §4.4's persistent benefit was measured under burst arrival at fixed concurrency, which loads the server differently from a steady arrival process at the same nominal rate.

For the fourteen-shape ladder this trade is favorable, since it costs no capacity at all: four additional compiled shapes reduce tail latency by up to 46% in the regime most interactive deployments run in. For the twenty-one-shape ladder, which costs 8.8% of capacity, the same arithmetic argues against it, since surrendering 8.8% of the cache to gain 2.6% in sustained throughput is an unfavorable exchange at saturation.

4.6 Prefix caching moves the target the ladder is placed against

Prefix caching is disabled elsewhere in this work, and production vLLM enables it by default. It removes already-computed prefix tokens from the prefill, so the step lands on a different compiled shape than the request's length implies, which bears directly on a recommendation about where to place entries.

Testing this requires a workload with a cacheable prefix. Requests built from independent token sequences share nothing, so a cache cannot hit them, and an experiment run against such a workload would report caching having no effect while saying nothing about production. Here every request shares a fixed 2048-token prefix and varies only its 952-token tail, which is the structure of a system prompt or a few-shot preamble. Prompt 3000, two concurrent requests (Table 16):

Table 14: Ladder difference with prefills isolated by a 120 ms arrival stagger. Prompt 300 pads to 512 on both ladders and measures the offset between server instances; 1200 pads to 2048 on both and should show no effect.

n	prompt 300 (offset)	prompt 1200, corrected	prompt 3000, corrected
4	-11.2 ms	+8.2 ms	**-65.8 ms**
8	-9.7 ms	+1.2 ms	**-108.1 ms**
16	-4.5 ms	+2.7 ms	**-98.1 ms**

Table 15: Sustained goodput and latency against offered load, both ladders at the stock memory fraction.

offered req/s	goodput (default → gap1024)	p50 ms	p95 ms
2	2.30 → 2.30	286 → 246	504 → 318
4	4.59 → 4.59	326 → 256	758 → 496
8	9.03 → 9.08	**974 → 529**	**1622 → 996**
12	12.45 → 12.68	2088 → 1895	3471 → 3093
16	13.45 → 13.84	2571 → 2275	3570 → 3418
24	**14.03 → 14.40**	3026 → 2921	3881 → 3727

Table 16: Ladder placement with and without prefix caching, over a workload sharing a 2048-token prefix.

ladder	caching	e2e	prompt tokens cached
default (10)	off	292.5 ms	0
gap 1024 (14)	off	256.6 ms	0
default (10)	**on**	181.5 ms	+43,008
gap 1024 (14)	**on**	178.5 ms	+43,008

Effect of prefix caching on the placement benefit The placement benefit falls from 35.9 ms, or 12.3%, to 3.0 ms, or 1.7%. A prediction registered before measurement anticipated this, and for the stated reason: with 2048 tokens cached the server prefills approximately 952, which pads to 1024 on both ladders, while the two ladders differ only at 3072 against 4096. The entry chosen in §4.3 is no longer the entry the step uses. The cached-token counter is the arm's own evidence that the treatment applied, since 43,008 is exactly 21 × 2048, so twenty-one of twenty-two requests hit the prefix and the first populated it.

The mechanism is unchanged and the operating point has moved. Caching does not make padded tokens cheap; it removes tokens from the prefill, and a ladder placed against prompt length is then placed against the wrong distribution. **Entries should therefore be placed against the distribution of uncached prefill lengths.** This also bounds §4.3 and §4.5, which are measured with caching disabled and so describe workloads with little prefix reuse. On this workload caching is worth considerably more than the ladder: 292.5 ms to 181.5 ms, a 38% reduction, against the 12.3% the best placement achieves without it.

4.7 A ladder can be chosen offline from a length distribution

§4.3's placement was chosen by knowing two prompt lengths in advance, which is an existence proof rather than a method. A method begins from a length distribution, selects a ladder without reference to latency, and is then correct.

We sampled 120 prompts from a lognormal distribution (median 1200, σ=0.9) and replayed the same lengths against every ladder, pairing the arms on workload. Each arm's latency was then predicted from its expected padded-token count multiplied by the 35 µs per padded token measured in §4.3:

Predictive accuracy of the offline model The offline model predicted the measured result to within 5%: 7.5 ms predicted against 7.1 ms measured. Inverting it gives 33.3 µs per padded token against the 34.9–35.3 µs of §4.3, and the two workloads share nothing, since §4.3 used two fixed lengths straddling known entries and this a heavy-tailed mixture over the whole ladder. A constant fitted on one workload and confirmed on another supports using the model for design.

Ladder design therefore does not require a hardware sweep: sample the length distribution, compute expected padding per candidate ladder, and multiply by the per-token cost.

Constraining the objective The objective must be constrained, and the unconstrained objective is the premise this paper refutes. Expected padding falls monotonically with shape count, so minimizing it alone drives the ladder toward the finest the stack can compile, which §4.3 shows the stack cannot afford. Both finer arms failed to boot at the stock memory fraction, the 35-shape ladder more decisively (23.75 MB requested against 4.94 MB free) than the 21-shape one (32.50 MB against 12.40 MB). The feasible set here was {10, 14} and the answer was 14. The configuration rule is to select the greatest shape density that satisfies the memory constraint, with entries placed against the length distribution.

Comparison against BucketServe's ladder objective The procedure above minimizes expected padded to-

Table 17: Ladders selected offline from a lognormal length distribution, with predicted and measured latency.

ladder	shapes	mean padded tok/req	predicted Δ	measured e2e	boots at 0.92?
default	10	602	—	219.8 ms	yes
gap 1024	14	389	-7.5 ms	**212.7 ms** (-7.1)	yes
gap 512	21	—	-16.2 ms	*does not boot*	**no**
gap 256	35	—	—	*does not boot*	**no**

kens. BucketServe specifies a different objective for the same decision, minimizing expected relative waste, `E[1 - S/U_b]`, and derives the condition that a bucket's upper bound should equal the conditional expectation of lengths within it. That condition is the Lloyd-Max centroid condition, and their paper declines to compute it, describing it as computationally expensive in practice. It is a local condition; we instead solve their objective globally by dynamic programming over a discretized length axis, which is `O(K N^2)` and takes milliseconds, so their objective is given a better solution than their own formulation proposes.

Neither ladder is expressible through `VLLM_TPU_BUCKET_PADDING_GAP`, whose family is "double while the doubling step is no larger than the gap, then step linearly". Both were compiled by patching `get_token_paddings` to accept an explicit ladder, which leaves the function unchanged when the variable is unset. Both ladders also carry fixed entries at 16, 32, 64 and 128: BucketServe's boundaries are derived from prompt lengths and begin at 208, and a ladder without small entries pads a two-token decode step to 208. Both arms therefore spend the same number of shapes, fourteen, and differ only in where the ten free entries sit.

The padded-token objective is faster by 1.61 ms, with a 95% interval of [1.04, 2.17] and $p < 0.001$ over nine replays per arm. Both ladder-design objectives beat the stock ladder by roughly 15 ms, so the disagreement between them is small next to the decision to design a ladder at all. The direction is what the cost model predicts: relative waste treats a 10-token overshoot on a 100-token request as equal to a 1000-token overshoot on a 10,000-token request, whereas §4.3 prices a padded token at a constant, so absolute padding is the quantity that converts to time.

Two honest qualifications. At three replays the same comparison gave [-5.39, +1.83] and did not resolve, and the arms were an order of magnitude noisier; the effect is real but small enough to need the replays. And the measured gap is about half the 2.8 ms the linear model predicts from 80 padded tokens, implying roughly 20 µs per token here against the 34.9–35.3 µs of §4.3. The per-token cost is therefore not constant across the range: the model is accurate where padding is large, which is where ladder design is decided, and overestimates the marginal value of removing padding once little remains.

Applying the procedure under traffic drift The procedure selects a ladder from a length distribution, and a deployment's distribution moves. Re-selection is cheap to decide and expensive to apply: the decision is arithmetic over a sample of recent lengths, while applying it requires a restart and a cold compile, measured here at 285 s for ten shapes. Re-selection is therefore a daily or weekly action rather than a continuous control loop.

The signal that should trigger it is already available to the server and costs nothing to maintain. A server knows both the compiled shape it selected for a step and the real token count in that step, so a running counter of their difference gives the realized padded share exactly, without sampling. Re-selection is warranted when that realized share exceeds what the offline model predicts for the ladder in use by more than the margin that would justify a restart. We note that `iteration_tokens_total` is not a substitute for this counter: §4.4 shows its power-of-two bins are coarser than the ladder spacing being compared. We have not built this controller, and report it as the natural operational form of the result rather than as a contribution.

One caveat bounds the model's reach. Predicting padding requires knowing the ladder a gap will produce, and the rule is not the obvious one: the stack continues doubling while the doubling step is no larger than the gap, then switches to linear spacing. At gap 256 this inserts an entry at 768, which a reading of "powers of two, then linear" does not predict. The predictions above were computed from the corrected rule, and each arm re-reads the ladder its server printed.

4.8 Supporting ablations and controls

A published latency predictor A published latency predictor, reproduced and scoped. LENS predicts NPU inference latency to 2.15% mean absolute percentage error (MAPE) using a per-bucket `intercept + slope` × `length` fitted from two end-to-end measurements per bucket. Reproducing its protocol on TPU across 5 buckets × 3 batch sizes, with 7 repeats per point and a mid-bucket point withheld from each fit, gives MAPE 5.23% and a worst cell of 22.4%, near-perfect at n=1–2 and failing at n=4. Replacing the model with a constant, the mean of the two calibration points:

At n=1–2 the within-bucket curve is nearly flat, at a flatness of 0.97, so any two-point fit is near-perfect. LENS does beat a constant there, at 0.38% against 0.96%, but

Table 18: Two ladder-design objectives at equal shape count, on the same replayed workload. Their objective minimizes relative waste; ours minimizes absolute padded tokens.

ladder	shapes	padded tok/req	mean e2e
stock	10	602	226.2 ms
gap 1024	14	389	215.6 ms
BucketServe objective	14	328	209.0 ms
padded-token objective	14	**248**	**207.4 ms**

Table 19: LENS against a constant-only model, held-out mean absolute percentage error.

batch size	LENS	constant-only
1	0.38%	0.96%
2	0.39%	0.86%
4	19.77%	**14.80%**

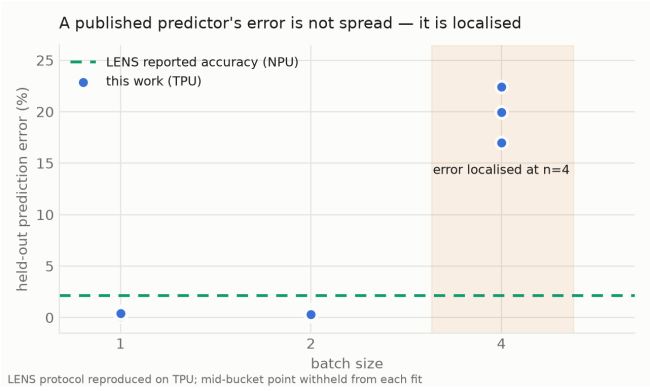

Figure 2: Held-out prediction error against batch size, with LENS's reported 2.15% as a reference line and the failure region shaded. The claim is not that the predictor is inaccurate; it is that the error grows sharply by n=4. Batch sizes 1, 2 and 4 are sampled, with the failure at the endpoint, so these data cannot say whether accuracy returns above 4.

the gap is 0.6 percentage points on errors already below 1%, and it is not evidence that the model form transfers. At n=4 the length term is actively harmful. The failure is not an artifact of which points were fitted: over all three choices per cell the n=4 error varies by up to 44.8 percentage points, but its minimum is still 16.97%, far above LENS's reported 2.15%. Batch sizes above 4 were not measured, because this experiment reproduces LENS's own protocol, which is specified over bucket structure rather than batch size.

Decode cost against batch size Decode cost against batch size. With `prompt_len=256` and `output_len=64`, measured across the full compiled request ladder:

There are two regimes and the boundary between them is sharp. Below n≈8 the step barely moves, with batch rising eightfold for 1.22 times the cost. Above n≈32 the step is nearly linear in batch, at 1.67, 1.82 and 1.86 times for successive doublings, and per-sequence cost flattens

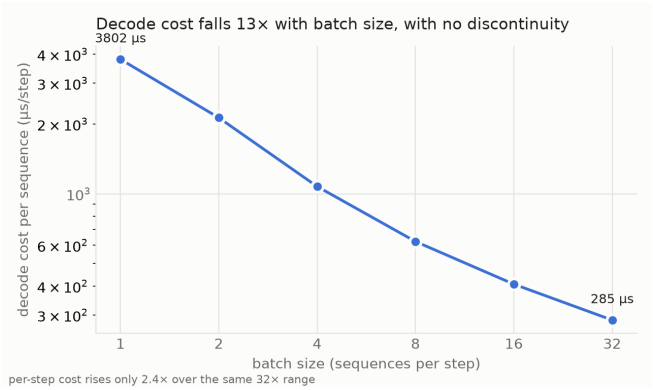

Figure 3: Decode cost per sequence against batch size, log–log.

at roughly 200 µs. Queue time at n=128 and n=256 means those columns are not clean wide batches and no claim depends on them.

Distribution of device time Where device time is spent. An operator profile is a direct observation, which the roofline is not. Share of TPU device time:

Matmuls dominate at low batch and give way to attention as key–value state grows, while collectives hold a flat 13.4% whose latency component no roofline models. Nothing moves discontinuously at n=4.

An invalid microbenchmark One microbenchmark does not measure the quantity it appears to. An isolated matmul at the model's real sharded shapes returns 142.9 µs at M=1 and 143.6 µs at M=256, a flatness that can be read as the weight-load floor with confounds removed. The qkv projection holds 7.86 MB per chip, so the bandwidth floor is 9.6 µs: the measurement sits fifteen times above it, at 7% of peak, and is timing per-dispatch overhead. Amortizing over a loop reports 1250% of peak because XLA hoists the loop-invariant matmul; chaining iterations is physically valid at 79% of peak but streams weights from HBM every iteration and remains bandwidth-bound.

Per-dispatch variance Per-dispatch variance is a prefill phenomenon. Over 9 repeats on the same server, decode spread is 1.00–1.04× at most batch sizes, while prefill is 1.00–1.03× at n≤4 and 1.18–1.26× at n≥8. Variance appears exactly where the scheduler begins splitting dis-

Table 20: Decode cost against batch size across the compiled request ladder.

n	1	8	32	64	128	256
ms/step	4.02	4.91	9.19	15.32	27.90	51.83
µs/step/sequence	4020	614	287	239	218	**203**
queue (ms)	0.0	0.0	0.0	0.1	62.2	298.3

Table 21: Share of TPU device time by operator category.

n	attention	collective	matmul/fusion
1	6.8%	13.5%	**78.5%**
4	15.4%	13.9%	69.6%
16	34.2%	13.4%	**51.4%**

patches, and decode, which makes no chunking decision, never exhibits it. This localizes the effect without explaining it. The aggregate figure does not describe every cell: bootstrapping the decode cells §4.1 depends on gives 95% interval widths of 38.7% at n=8 and 28.2% at n=9 over 21 repeats, wider than several differences a reader might otherwise treat as signal.

Sharding ablation Free request padding is not an artifact of the sharding. Model, chips and workload were held fixed while tensor-parallel degree alone was varied. The prediction was registered before measurement: per-chip weight bytes scale as 1/TP, so the level should scale with 1/TP and the shape should be preserved.

Both halves of the prediction were wrong. The level scales sub-proportionally, which the roofline cannot explain because it does not model the inter-chip collectives the higher-TP arms pay, and the curve becomes flatter with less sharding rather than preserving its shape. Both errors follow from a single omitted term. Fitting `T(TP) = W/TP + F`, a weight load that shards plus a fixed cost that does not, gives `T(TP) = 2.48/TP + 0.38` normalized to the TP=4 step, with fitted values 1.00, 1.62 and 2.86 against measured 1.00, 1.63 and 2.86. Fixed, non-sharding cost is 38% of the TP=4 step, roughly 1.5 ms at n=1, and that single term explains both the sub-proportional level and the flattening. The fit has two free parameters against three points, one of which is fixed by normalization, so it carries a single genuine degree of freedom and the residual overstates how much evidence it provides.

`F` is not the collectives: a constant term cannot represent inter-chip communication, which is zero at TP=1 and largest at TP=4 and therefore scales with TP, whereas `F` by construction does not. The ablation nonetheless answers its question: if request-dimension padding were cheap only because that dimension is not the bottleneck at TP=4, reducing TP would expose it, and instead padding is cheapest at TP=1 where the fixed cost is largest.

Model scale, and dtype as the lever Model scale does not move the regime map. A second registered prediction failed here. TinyLlama-1.1B has a 3.6-fold smaller per-chip weight floor than Qwen3-4B, so if the mechanism were that floor its paid share should be higher. It is lower, at -1.2% and 13.4% at n=4 and -1.1% and 5.9% at n=8. Model size is the wrong lever: for dense weight-stationary decode, bytes $\approx 2\cdot$params and FLOPs per token $\approx 2\cdot$params, so both models sit at 1.00 FLOP/byte/-token and arithmetic intensity is the batch size, independent of parameter count. The ridge is a property of the chip (v5e: 197 TFLOP/s ÷ 819 GB/s ≈ 240 FLOP/byte), and shrinking the model shrinks the floor and the work in the same proportion. This answers the single-model limitation analytically for that regime, and breaks for mixture-of-experts models, where bytes scale with distinct experts touched, and at long context, where key–value state rather than weights sets the floor.

Two cautions apply. The identity is derived for decode, while the paid-share numbers are prefill; applying a decode argument to prefill measurements crosses between two of the quantized dimensions, which §3 states this paper does not do, so the numbers are reported as measurements with the explanation withdrawn. The intensity argument also explains why model size should not move the paid share, not why it moved downward; the likely reason is that non-weight fixed costs form a larger fraction of a 0.55 GB model's step.

Dtype is the lever that should move the token dimension. W8 weights halve bytes and leave FLOPs unchanged, doubling intensity per token and moving the crossing from batch ≈ 240 to ≈ 120. That yields a registered prediction — under W8 the token-dimension paid share at a fixed boundary rises — which **has not been run**. It is future work rather than a result, and nothing above depends on it.

4.9 The request-dimension result replicates on GPU

§4.1's request-dimension finding was measured on TPU. Whether it holds on another architecture is testable rather than assumed: same vLLM 0.25.0, same instrument, an L4 (23 GB, TP=1) in place of the v5e:

The increments are the measurement; the levels are not. Eager execution pays a constant per-operation launch overhead, so the levels differ by roughly 9.3 ms throughout, and that constant cancels in any increment. It cancels to within 9 µs on the 8→16 step, at 1.693

Table 22: Tensor-parallel ablation. Both halves of the registered prediction fail, and a single omitted fixed term explains both.

TP	per-step level vs TP=4	predicted	cost rise, n=1→32
4	1.00×	1.00×	2.33×
2	1.63×	2.00×	2.41×
1	2.86×	**4.00**×	**1.83**×

Table 23: GPU control on an L4: CUDA-graph capture against eager execution. The increments are the measurement; the levels differ by a constant launch overhead.

arm	n=8	n=9	n=16	8→9	8→16	startup
CUDA graphs on	10.605	10.887	12.298	**0.283**	**1.693**	118.7 s
`-enforce-eager`	19.934	20.150	21.618	**0.215**	**1.684**	10.7 s

against 1.684 ms, which is the control this comparison requires: the two arms measure the same underlying work plus an offset, so their difference isolates what capture adds.

On that basis, padding a batch from 8 up to the captured entry at 16 costs approximately **67 µs**, being the 0.283 ms increment with graphs against 0.215 ms without. That is 0.6% of a 10.9 ms step, or 4.0% of the nominal padding implied by rounding 9 up to 16.

This tests one of the three dimensions, and it is the one least in doubt. vLLM's CUDA path captures a graph per batch size, so what these arms vary is the request dimension, which the TPU results explain with a data-structure argument (§4.1) and where no proposed optimization was going to pay. The token dimension, where this paper locates the only surviving effect, is not reached: no arm here varies tokens per step at fixed batch. The cross-architecture statement this table supports is therefore that **request-dimension padding is close to free on both architectures**, not that the premise is false on both. Whether a GPU stack pays for token padding as a TPU stack does is untested, and is the experiment that would make the cross-architecture claim general.

The comparison is also a bound rather than an equality. §4.1's TPU statistic carries intervals of roughly ±50 percentage points, so this table cannot resolve a difference between the architectures; what both support is that the paid share is small on each. These remain single measurements: three batch points, one GPU, no repeats.

What is paid is the capture, and it is paid at startup. Enabling graphs costs 108 s of initialization, 118.7 s against 10.7 s, for a capture set fixed in advance. That is precisely the quantity BucketServe and LAPS manage when they write that the number of graphs must be limited, and it is a warmup cost. The TPU analogue is XLA compilation: 5–30 minutes for the first bucket and 30–120 s per additional one.

5 Optimizations designed against these measurements

Five interventions were designed against the compiled-shape premise. Each rejection below identifies a dimension the measurements show carries no cost:

- **Ladder placed against the workload** — works: -12.1% at n=2 and -46% p50 below the knee, at full memory; beats BucketServe's objective by 1.61 ms at equal shape count; -1.7% once prefix caching is on (§4.3–§4.7).
- **Bucket-aware admission control** — premise false (§4.2).
- **Ladder redesign on the request dimension** — D1 does not exist; D3 inert by default.
- **Last-chunk decomposition** — 20.6% worse measured (51.06 vs 42.33 ms).
- **Bucket-aligned step packing** — implemented twice: inert, then output-corrupting.

The one that works targets the single dimension the measurements leave open, and the rejections are informative rather than incidental. Three of them address the request dimension or per-request length padding, where §4.1 and §4.2 find no cost to recover, so their failure is what those sections predict; the fourth restructures work the stack already packs. The token dimension is the only place where the premise was not refuted by measurement, and the only place where an intervention produced a gain — though not for the reason expected, since the payoff was predicted to be confined to low concurrency and was not (§4.4).

A second positive measurement — release timing saving 26% of TPU time against stock at 25 req/s (p=0.001, six paired seeds) — is dynamic batching rather than a shape effect, and is reported as a re-measurement rather than as a contribution.

It is a low-load effect that reverses. Swept across arrival rate against stock, positive being a saving:

Table 24: Release-timing policies against arrival rate, positive being a saving in TPU time against stock.

req/s	wait	hybrid	hybrid p95
10	+36.2%	+29.0%	-32.6%
25	+22.2%	+22.8%	+28.9%
40	+7.1%	+17.6%	+21.5%
55	+7.2%	+11.6%	-6.6%
70	+2.3%	**-11.7%**	+18.7%

The saving decays monotonically with load and becomes an 11.7% penalty by 70 req/s. A single measurement at 25 req/s cannot distinguish that from a robust effect, and it samples the most favorable region of the curve.

It is also not free. The harness scrapes `/metrics` around every batch, adding a median 22.6–24.9 ms of inter-dispatch overhead to a policy that never waits. That inflates stock's p95 from roughly 24 ms to 86 ms and conceals the delay the waiting policy introduces deliberately. Measured under that harness the cost appears as +14.8% p95 (p=0.570); simulated on an efficiently driven server the same policy costs approximately +188% p95 at 25 req/s. The defensible statement is narrower: the hybrid policy reaches nearly all of wait-to-fill's saving, at 30.2% against 31.9%, for a small fraction of its latency cost, at +188% against +1461%. The result is a point on a cost–latency trade-off curve.

The cost model does not survive a wider sweep. Holding out rates it was never fitted on gives 4.9% MAPE overall but a worst cell of 19.7% (hybrid at 55 req/s), against this project's own 15%-per-cell rule, so it fails. The earlier holdout varied neither rate nor prompt length and so could not detect an error constant across them. The simulated policy numbers are internally consistent predictions that do not transfer to unseen load.

Bucket-aligned packing requires a separate note. The second implementation measured -29% TPU time and -49% p99; the correctness gate then showed that 4 of 48 greedy completions differed, every one at a prompt length just above a bucket boundary. The patch was silently dropping prompt tokens. Trimming `num_scheduled_tokens` after the scheduling loop leaves the request's bookkeeping untouched, so deferred tokens are skipped rather than rescheduled, and the step is cheaper because it does less work.

6 Validation, and what it does not cover

Every reported quantity is produced under a configuration contract that aborts a run when a controlled variable is left undeclared, is tied to a run identifier, and is recomputed from captured data by a script that exits non-zero if it disagrees with the text. §2 states the registered-prediction requirement that accompanies it.

No equivalent validation applies to explanations. A proposed mechanism can be written, accepted, cited by later sections, and carried across successive drafts without any stage of the process being able to contradict it. Two instances in this paper show the consequence. The bandwidth account of free request padding persisted through four sessions of work before §4.1's utilization measurement refuted it, and it was never supported by evidence so much as never confronted with any. The control cell of §4.4 was chosen on the reasoning that a 300-token prompt pads identically on both ladders, which is true only while each request prefills alone; one scrape of the executed step distribution settled it, after both arms had run.

The consequence for a reader is that the measurements in §4 and the mechanisms offered for them do not carry the same weight, and we present them accordingly. Two qualifications keep that statement honest.

First, survival is not uniform. §4.2's paid-share table survived the discovery that its rows were measured over different boundary sets as an *ordinal* claim; the cardinal values did not. §4.4's rows above n=2 survive as floors rather than estimates under burst arrival, and only the isolated-dispatch control recovers interval-level statements. A value that reproduces while its interpretation is withdrawn has not survived in the sense the word usually carries.

Second, the asymmetry is partly a property of what can be checked rather than of what is true. A measurement has a value a script can recompute; a mechanism does not, so it is never submitted to the check at all, and passing no check is not the same as passing one. Our own record limits the stronger reading: four of the errors we caught were instrument-definition failures, in which an analysis measured something other than its target, and those corrupt numbers rather than explanations. No guardrail here covers that class, so we cannot bound residual error in the measurements either.

7 Limitations

One TPU slice, one GPU, one primary model. A v5litepod-4 with a 4B model, and a single L4 for the control. The sharding objection is answered by the TP=1/2/4 ablation (§4.8), which finds request padding cheap at every sharding, but model scale and multi-host topology are unmeasured, and both change the fixed costs within which padding can be absorbed. §4.8 argues analytically that the regime map is a function of batch size and dtype rather than parameter count, for dense weight-stationary decode only.

The strongest single number is the least well supported. The ~85% paid share at n≤2 rests on one

boundary and carries no interval (§4.2).

Prefix caching is measured but not swept. §4.6 tests it at one prefix length, 2048 of 3000 tokens, on one workload shape. This is enough to show that the placement target moves and not enough to say where it lands for a given amount of prefix reuse. Everything outside §4.6 is measured with caching disabled and so describes workloads with little prefix sharing.

No production trace. §4.2's four length distributions are parametric families matched on mean length, not a trace of real traffic. They establish that padded share varies by a factor of four with distribution shape at equal offered tokens, and that dispersion does not order it, but the figures belong to those families rather than to any deployment. §4.6 further shows that any such figure depends on how much prefix reuse a workload has, since caching changes the length actually prefilled.

The ladder benefit has no measured upper bound in concurrency. §4.4 sweeps 1 to 16 and finds no crossing, so we cannot say where a finer ladder ceases to pay, only that it has not ceased by 16. Above n=4 that experiment has no valid placebo, so those rows are floors on the effect rather than unbiased estimates.

The recommendation is mediated by version-pinned internal flags. `VLLM_TPU_BUCKET_PADDING_GAP` and `ATTN_BUCKETIZED_NUM_REQS` are internal to vLLM and `tpu-inference` 0.25.0, and a scheduler or compilation refactor upstream could change the ladder they produce, or remove them. The measurements would survive such a change, since they characterize a mechanism rather than an interface, but the operational advice is pinned to this version. §4.7's rule mitigates this only partly: it states which ladder to want, while the flags are how one currently asks for it.

Cost per padded token on a mixture-of-experts model is unmeasured. No slice was available: `v5litepod-4` capacity was refused in thirteen zones across three continents, on both on-demand and spot pricing, over a full day of half-hourly attempts. The obstacle is capacity rather than budget or design.

What the implementation settles is the direction. In `tpu-inference` 0.25.0 a padded token entering the fused expert kernel carries whatever the routing top-k returned for its unwritten row, and is dispatched to that many real experts — eight, for Qwen3-30B-A3B. Two costs follow, and only one has a dense analogue. The first is one token's worth of expert compute, which is what a real token pays too, and which therefore leaves the paid *share* unchanged, that share being a ratio. The second is that a padded token can activate an expert no real token in the step required, adding a group to the grouped matmul that would not otherwise run. That second term grows as the batch shrinks, which is exactly where padded share is already highest (§4.2), and it is the quantity §4.8's arithmetic-intensity argument already names as the reason that argument does not extend to these models: bytes there scale with distinct experts touched, and padding is a way of touching more of them. The backend carries a flag, `MOE_ROUTE_PADDING_TO_EXPERT0`, that collapses padding onto a single expert at zero weight; it ships disabled, is inert when attention is data-parallel, and fails open, serving the unmitigated path if it cannot read the step's valid-token count.

The sign survives this; the magnitude does not. Whether the second term is three percent of a padded token's cost or sixty decides whether that flag is an operational recommendation or a footnote, and no reading of the source answers it.

A published diagnosis of the `gpt-oss-20b` failure was wrong. An earlier draft attributed its refusal to initialize at TP=4 to a parameter axis of size six. No axis of that size appears in its configuration. Its weights are MXFP4, whose 4-bit values are packed in blocks of 32, and $2880/4 = 720$ is not a multiple of 32, so a block would straddle two chips. TP=2 divides cleanly but cannot run on this hardware: the device mesh is built as (data, tensor) over every visible chip, so the product of the two must equal the slice's four, and any data-parallel degree above one disables the flag above. With no two-chip topology offered and single-chip quota at zero, that model has no configuration available to it here. `llama4` remains out of reach at 109B parameters.

The GPU control is inference-tier hardware. The L4 has neither the compute throughput nor the memory bandwidth of the accelerators most production serving runs on, and the arithmetic-intensity ridge sits at a different point there than at v5e's ~240 FLOP/byte. Since the ridge is what sets the batch size at which token padding stops mattering (§4.8), the batch thresholds reported here should not be transferred to other hardware without re-measurement.

Prefill step cost above n=16 is not isolable, and at n=16 the clean sample is 7–11 dispatches per arm.

The n=4 convergence is unexplained and is not an operator effect. An operator-level profile shows every category of device time moving smoothly through n=4. The three observations that appear to converge there are not independent: LENS's failure and the paid-share drop are the same quantity described twice.

The GPU control is one point rather than a curve. Startup was measured at vLLM's default capture set, and the number of captured shapes was not varied, which is precisely the axis BucketServe and LAPS trade along.

Co-located prefill and decode only. On a disaggregated deployment the padding question divides in two,

and §4.8's finding that variance is a prefill phenomenon is the asymmetry that motivates disaggregation.

8 Related work

Pope et al. derive the memory-bound to compute-bound transition for transformer inference on TPU analytically; §4.1's byte accounting is a measured instance of that regime rather than a discovery. We do not claim that the weight-load floor explains free request padding, since §4.1 tests that account and rejects it.

RPA is the technique this work validates: per-request padding costing nothing is what its ragged-tiling design predicts, though it does not discuss the request-count dimension, report cost against batch size, or quantify how much padding survives it. **LENS** (§4.8) supplies the model form; we supply the hardware it was not tested on and the ablation showing that its length term is not what carries its reported accuracy.

PagedAttention/vLLM is the stack measured throughout; **Orca**'s iteration-level scheduling produces the per-step batches; **Sarathi-Serve** introduced the chunked prefill that §4.2 controls for. **DistServe** and **Splitwise** disaggregate prefill from decode, which is the architectural response to the finding that variance is a prefill phenomenon, and which bounds our advice to co-located deployments. **Vidur** established simulator-fidelity validation as the standard for simulation-based serving studies; our holdout discipline follows it.

SGLang and **TensorRT-LLM** implement their own shape and graph handling, with different capture policies and, in TensorRT-LLM's case, an ahead-of-time engine build with explicit optimization profiles. Nothing here is measured on either, and the claims should be read as applying to vLLM-style designs, in which shapes are compiled or captured from a ladder the serving loop rounds up to.

BucketServe and **LAPS** manage length-bucketing overhead on GPU. §4.7 runs BucketServe's own ladder-design objective on this stack rather than arguing against it: solved globally and given the same shape budget, its ladder is 1.61 ms slower than one chosen to minimize absolute padded tokens, and both are about 15 ms faster than the stock ladder. Their ladder design is therefore effective, and our disagreement with these systems is narrower than a premise-level objection. It concerns which dimension the padding occupies — the request dimension is free (§4.1) and per-request length padding does not exist (§4.2) — and, for the token dimension where it is real, which objective converts to time.

The cross-architecture comparison itself — same vLLM 0.25.0, same instrument, an L4 in place of the v5e — is measured rather than asserted, and is reported as a result in §4.9 rather than as related work: it finds request-dimension padding close to free on both architectures, with the same caveats of scope (one GPU, no repeats) that apply to the TPU measurements it parallels. Enabling CUDA-graph capture there costs 108 s of initialization, which is precisely the quantity BucketServe and LAPS manage when they write that the number of graphs must be limited, and it is a warmup cost — the TPU analogue is XLA compilation, at 5–30 minutes for the first bucket and 30–120 s per additional one.

How, then, do prior bucketing techniques achieve their reported speedups? If the padding premise is false on both architectures, systems reporting end-to-end improvements from bucketing are improving something else, and §5 supplies a candidate from our own data. A positive measurement in this work — release timing saving 26% of TPU time at 25 req/s — is dynamic batching, an arrival-and-composition effect rather than a shape effect. Bucketing schemes change which requests occupy a step together, and that is worth something independent of padding. We place weight on the mechanism rather than the magnitude, because the magnitude is the weaker half: the saving costs latency once the harness overhead inflating the baseline is removed, and it reverses at high load (§5). The defensible claim is the negative one — **a bucketing result that does not control for batch composition cannot distinguish a shape effect from a scheduling one** — and it rests on the premise measurements of §4.1 and §4.2.

9 Conclusion

Two accelerator families reach the same design by different routes: XLA compiles a ladder of shapes, and CUDA captures a graph per batch size. Both round every step up to the nearest entry, and the optimization literature treats that rounding as a cost to be recovered. It is not, on either. A batch just above an entry costs what the entry below costs, because a ragged attention kernel does almost no work for slots holding no key–value blocks — under 0.7 µs per slot, against 27.5 µs if it were paid — and a captured graph does not care that part of its batch is unused.

What shape coverage costs is a boot-time budget, and only part of it amortizes. Enabling CUDA-graph capture costs 108 seconds of startup, and XLA compiles the first TPU bucket in 5–30 minutes; a persistent compilation cache erases most of that on every restart after the first (§4.3). What does not amortize is memory headroom: a finer ladder can raise the fraction the compiler needs before the server will start at all, at the cost of key–value capacity, and no cache reuse fixes that (§4.3). That is the quantity BucketServe and LAPS manage when they write that the number of graphs must be limited, and it is a startup and memory-footprint budget rather than

a throughput one. Reducing the number of shapes is worth doing for time-to-serve and resident executables. Routing requests to avoid run-time padding is not worth doing.

The exception is the token dimension, where padded tokens are real arithmetic: the paid share is 23.1% of nominal at batch 4, falls to indistinguishable from zero by 16, and is around 85% at batch ≤ 2 — a figure resting on a single boundary with no interval, and the least well supported number here. That low-batch regime is interactive serving, tight-latency deployments, and the prefill half of any disaggregated system, and it is where the ladder buys something.

The variable that pays is placement, not shape count. A fourteen-shape ladder adding a single entry the default lacks reduces end-to-end latency by 12.1% at the prompt length that straddles it, gains nothing at a length it does not, and boots at the stock memory fraction with unchanged cache capacity. A twenty-one-shape ladder achieves the same reduction and will not start above a memory fraction of 0.85, costing 8.8% of cache capacity and 53% more startup. Every cost measured scales with cardinality. The ladder can be chosen offline: expected padding computed from a length distribution predicted the measured reduction to within 5%, so the design rule is the finest ladder that still boots, with entries placed against the distribution the server actually prefills.

Two conditions bound that advice. The gain is a latency reduction below saturation rather than additional capacity: median latency falls 46% just below the knee, while sustained throughput rises 2.6% at it. And prefix caching, which production vLLM enables by default, reduces the gain from 12.3% to 1.7% by shortening the prefill onto a different compiled entry, so entries must be placed against uncached prefill lengths rather than prompt lengths.

Three predictions registered before measurement failed, and each failure was worth more than a confirmation. That record is the paper's methodological result: every reported measurement has survived revision while four of the last five retractions were claims about mechanism, because a measurement passes through machinery that can reject it and an explanation does not. The remedy adopted here is to require that a mechanism emit a falsifiable number before hardware is provisioned. It is cheap, and it is the practice we would most recommend carrying into other measurement work.